

# Resumo scripts v2

## 1. Tratamento de Erros ( `1_tratamento_de_erros.py` )

### Problema:

O registro de erros das equipes de campo era feito em arquivos de texto soltos e sem padrão. A consolidação manual desses dados em planilhas era lenta, sujeita a falhas de digitação e dificultava a análise das falhas frequentes.

### Como foi resolvido:

Um script automatizou a leitura e a padronização desses arquivos, inserindo os registros diretamente na base oficial. O sistema também passou a alimentar um dashboard interativo para análise rápida dos erros e das equipes com mais apontamentos.

## 2. Verificação de Posição de Coleta

### ( `2_verif_posição_de_coleta.py` )

### Problema:

A auditoria da posição de coleta das amostras era feita linha por linha para garantir que o realizado em campo batia com o Plano de Amostragem oficial. Esse processo manual era demorado e suscetível a erros de conferência.

### Como foi resolvido:

A solução automatizou o cruzamento dos dados de campo com o plano oficial. O sistema gera uma planilha auditada com alertas visuais para divergências e já indica a posição correta, agilizando os ajustes.

## 3. Separação de Fichas Duplicadas

### ( `3_separação_fichas_duplicadas.py` )

### Problema:

Pastas com centenas de arquivos de campo frequentemente recebiam PDFs repetidos referentes à mesma amostra física, gerando desorganização sistêmica e risco de duplicidade de informações.

### Como foi resolvido:

O script passou a ler o conteúdo interno dos PDFs para identificar o código real da amostra. Ao detectar duplicatas, ele isola automaticamente os arquivos repetidos em uma pasta de descarte, mantendo a base principal limpa e única.

## 4. Renomeador de Fichas Eletrônicas

### ( `4_renomeador_fichas_eletrônicas.py` )

### Problema:

As fichas de coleta eram salvas com nomes muito longos, fora de padrão e com textos

desnecessários, o que dificultava a ordenação e a busca rápida dos arquivos no dia a dia.

#### **Como foi resolvido:**

A ferramenta automatizou a renomeação em lote dos arquivos. Ela remove todos os textos desnecessários e mantém apenas o código oficial da amostra no título, otimizando a organização e as buscas em segundos.

### **5. RCE Reparação ( `5_rce_reparação.py` )**

#### **Problema:**

A conferência da calibração e verificação dos equipamentos exigia análise manual minuciosa para confirmar se os técnicos haviam seguido corretamente todas as etapas e a ordem química exigida, demandando muito tempo.

#### **Como foi resolvido:**

Um script automatizou a leitura dos relatórios para validar a sequência dos padrões utilizados pelas equipes. O sistema atualiza o controle geral e emite alertas automáticos sinalizando onde houve falha na ordem ou falta de calibração.

### **6. Relatório NTU ( `6_relatório_ntu.py` )**

#### **Problema:**

A atualização diária do relatório de turbidez era extremamente repetitiva. Exigia extração manual, alinhamento de colunas e uso de fórmulas complexas para cruzar dados e alimentar a base histórica sem quebrar o layout.

#### **Como foi resolvido:**

A solução automatizou o cruzamento de informações e a filtragem dos dados relevantes. O script localiza automaticamente onde colar as novas informações na base histórica, mantendo fórmulas e formatação intactas.

### **7. Extrator de Fotos ( `extrator_fotos.py` )**

#### **Problema:**

Extrair fotos de pontos de amostragem no meio de extensos relatórios em PDF exigia busca e salvamento manual. Era um processo muito lento e sujeito a erros, como a extração indesejada de logos e assinaturas de rodapé.

#### **Como foi resolvido:**

O script automatizou a busca varrendo os PDFs em lote. Utilizando mapeamento de seções de texto e filtros de tamanho, ele salva apenas as imagens válidas das amostras em pastas devidamente nomeadas, ignorando totalmente elementos de layout.

### **8. Sistema de Monitoramento de Agendamentos**

#### **Problema:**

O gerenciamento de agendamentos de coletas era descentralizado e totalmente manual.

Cruzar bases, calcular prazos de urgência e redigir mensagens individuais para cada contato tomava muito tempo e gerava falhas de comunicação.

**Como foi resolvido:**

Foi desenvolvida uma plataforma centralizada que sincroniza os dados e categoriza as pendências por nível de urgência. Com apenas um clique, o sistema redige as mensagens de agendamento já agrupadas e salva todo o histórico da operação.

## **9. Pipeline Cloud de Conferência de Fichas**

**Problema:**

Processar o recebimento de grandes volumes de fichas fotográficas compactadas demandava muito esforço computacional e era difícil de escalar. O processamento manual e pesado causava travamentos constantes na máquina.

**Como foi resolvido:**

A operação foi levada para a nuvem. O novo sistema recebe e descompacta os arquivos em grandes lotes automaticamente, aplica leitura inteligente para validar os dados das fichas e consolida tudo de forma rápida e segura em um dashboard de acompanhamento.

## **10. Automação de Criação de Grupos**

( `automação_criação_grupos.py` )

**Problema:**

A montagem de grupos de amostras e controle de qualidade para as equipes de campo era feita manualmente, exigindo copiar dados de bancos e reescrevê-los em bloco de notas para organizar as tarefas por data e equipe.

**Como foi resolvido:**

O fluxo foi totalmente automatizado. A solução lê a base de dados, correlaciona as amostras pertinentes e exporta os arquivos estruturados por data e equipe, prontos para uso em campo e já com alertas sobre dados faltantes.

## **11. Automação da Programação Diária**

**Problema:**

O envio da programação de campo exigia consolidar dados de diversas fontes diferentes manualmente. A necessidade de realizar cruzamentos e conferências visuais antes de enviar os cronogramas atrasava muito a operação diária.

**Como foi resolvido:**

Um sistema passou a integrar e consolidar automaticamente as múltiplas fontes de dados. Ele valida informações, remove pontos cancelados e gera as mensagens prontas para disparo, exigindo intervenção humana apenas para revisar casos de alerta.